

Design a Web Crawler - Robustness

1. What is Robustness?

- Robustness ensures the crawler:
 - Works correctly under failures
 - Handles errors gracefully
 - Continues operation without crashing

Critical for **large-scale systems**

2. Techniques to Improve Robustness

2.1 Consistent Hashing

Purpose

- Distribute load among multiple downloaders

How It Helps

- When servers are:
 - Added
 - Removed

System redistributes load **without major disruption**

Benefits

- Better load balancing
- High scalability
- Minimal re-distribution of data

2.2 Save Crawl States and Data

Purpose

- Prevent loss of progress during failures

How It Works

- Store:
 - Crawl states
 - Downloaded data

Benefits

- System can:
 - Restart from last checkpoint
- Avoids:
 - Re-crawling from beginning

2.3 Exception Handling

Problem

- Errors are common in large systems:
 - Network failures
 - Invalid responses
 - Server crashes

Solution

- Handle exceptions gracefully

Behavior

- Detect error
- Log error
- Continue execution

Benefits

- Prevent system crash
- Maintain stability

2.4 Data Validation

Purpose

- Ensure correctness of data

What to Validate

- Downloaded HTML

- Extracted URLs
- Response formats

Benefits

- Prevent invalid data from entering system
- Avoid downstream errors

3. Key Takeaway

Robustness ensures:

- Fault tolerance
- System stability
- Continuous crawling

Achieved using:

- Consistent hashing
- State persistence
- Exception handling
- Data validation